



Curso Seguridad del Software

por Adriana Rojas



CÁMARAS
ENCENDIDAS
POR FAVOR



DECLARACIÓN DE USO ÉTICO DE INTELIGENCIA ARTIFICIAL Y FUENTES EXTERNAS

- Se **autoriza y fomenta** el uso de herramientas de Inteligencia Artificial Generativa (como Gemini, ChatGPT, Claude, etc.), así **como del material complementario** proporcionado en clase, como apoyo para la fase de investigación, estructuración y generación de ideas. Sin embargo, el uso de estas herramientas está sujeto a las siguientes condiciones obligatorias:
 - **Responsabilidad y Autoría:** La IA puede generar alucinaciones (datos falsos o códigos erróneos). Es responsabilidad absoluta del estudiante verificar la veracidad técnica de las respuestas.
 - **Prohibición del "Copy-Paste":** No se aceptarán respuestas que sean una transcripción literal de la IA.
 - **Transparencia (Anexo de Prompts):** En caso de haber utilizado IA, se debe incluir un **Anexo** al final del documento que contenga:
 - La herramienta utilizada (ej. Gemini 1.5 Pro).
 - Copia íntegra de los **prompts** (instrucciones) enviados a la herramienta. Y los cambios hechos a dichos prompts para obtener una respuesta satisfactoria.

TEMA 1: IMPORTANCIA DE LA SEGURIDAD DEL SOFTWARE



INTRODUCCIÓN: SEGURIDAD DEL SOFTWARE DESDE LA MIRADA DEL DESARROLLADOR

- El código que escribimos define el nivel de riesgo de toda la organización.
Cada decisión de diseño impacta directamente la **Confidencialidad, Integridad y Disponibilidad (CIA)**.
- La seguridad no es una fase final: es un requisito funcional del software.

01

Confidencialidad – “Protege los datos por diseño”

02

Integridad – “Que el software haga solo lo que debe hacer”

03

Disponibilidad – “Software resiliente y confiable”

“Cada línea de código puede ser una puerta o un escudo.”





1 Década de 1960: Conciencia inicial de errores y control de acceso

Primeros debates sobre confiabilidad y permisos



2 Década de 1970: Modelos formales y criptografía aplicada

Nacen modelos de seguridad y uso práctico de cifrado



3 Década de 1980: Seguridad en redes y control de acceso fino

Protección frente a amenazas remotas y privilegios



4 Década de 1990: Seguridad en la web y políticas organizadas

Aparecen firewalls, políticas y pruebas de penetración



5 Década de 2000: Desarrollo seguro y normas de calidad

Integración de seguridad en ciclo de vida del software



6 Década de 2010: DevSecOps y automatización de pruebas

Seguridad como responsabilidad compartida y automatizada



7 Década de 2020: Zero Trust y enfoque en la triada CIA

Confidentiality, Integrity, Availability como guía



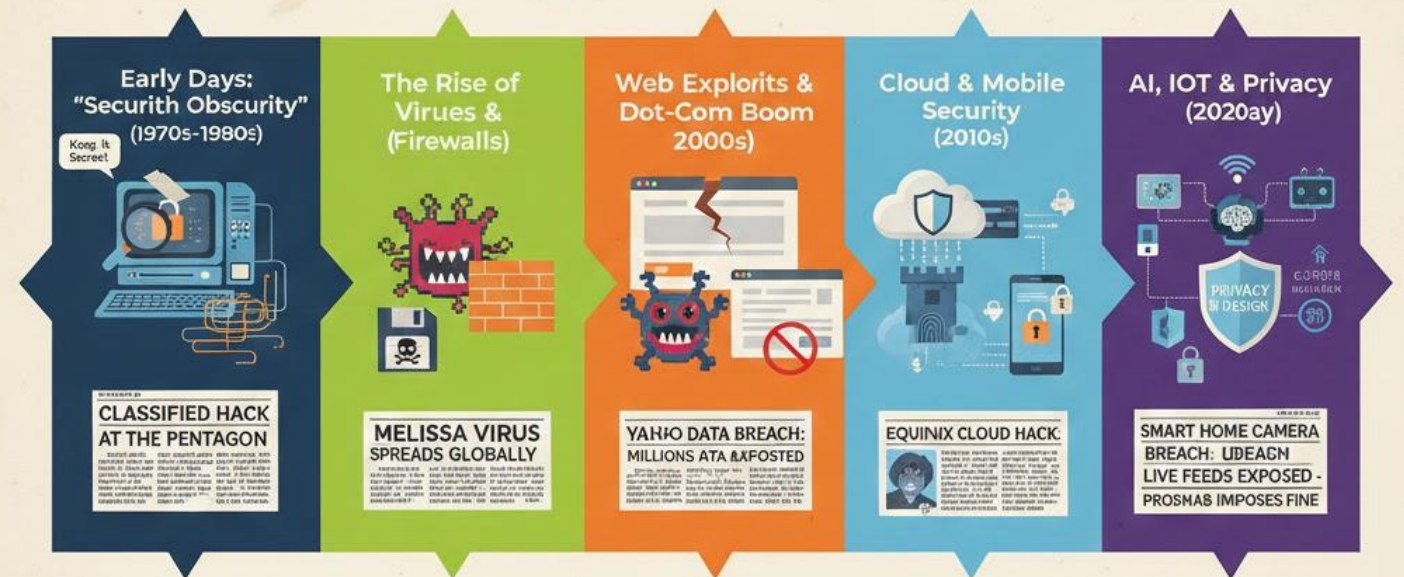
8 Resumen: De correcciones reactivas a prácticas proactivas

La seguridad pasó a integrarse desde el diseño

HISTORIA DE LA SEGURIDAD DEL SOFTWARE

History Software Security

- Vamos a buscar una noticia relacionada con cada una de los ciclos históricos de la Seguridad del Software
- Comentamos la noticia en clase
- https://miro.com/welcomeonboard/d3RocnBTQ1cwU3kyNjk3ekNzRUFqZW5Tc284c2ZjUDRVMndTOW5BcUZRV0pRdzRuUDFzTmZLY0hLdjUyN1VtdUJ2ZU5PZzB6dU1KWl9uemxCcXhMSXJ0N2JpSWlpL2lsK09mc3JBalcZWmsveG1vS2c2djRWeS91R3dJNW9mTVR6NG9oa1c3QWRybDJDeW5vVUVHSkZ3PT0hdjE=?share_link_id=318922984808



¿QUÉ ES LA SEGURIDAD DEL SOFTWARE?

- Es la disciplina de la ingeniería de software que comprende el conjunto de prácticas, principios, metodologías y controles técnicos aplicados durante todo el ciclo de vida del desarrollo de software (SDLC) con el objetivo de proteger un sistema frente a amenazas, vulnerabilidades y ataques maliciosos, garantizando la confidencialidad, integridad y disponibilidad de los datos y recursos que gestiona.

COMPLEJIDAD, EXTENSIBILIDAD Y CONECTIVIDAD



COMPLEJIDAD



La complejidad es el enemigo natural de la seguridad. Se refiere a la cantidad de partes móviles, líneas de código, capas de abstracción e interdependencias dentro de un software.

- El Riesgo:** A mayor complejidad, más difícil es predecir cómo se comportará el sistema ante un ataque. Las vulnerabilidades suelen esconderse en las "grietas" entre diferentes módulos.

- Principio de Seguridad:** Se busca la **Simplicidad**. Un código limpio y minimalista es más fácil de auditar, probar y proteger.

- En resumen:** "Lo que no está ahí, no puede ser hackeado".

EXTENSIBILIDAD



Es la capacidad del software para permitir el crecimiento o la adición de nuevas funcionalidades (plugins, módulos, APIs) sin alterar la estructura base.

- **El Riesgo:** Si el sistema es muy abierto, un atacante podría insertar código malicioso a través de una extensión no verificada. El desafío es permitir que el sistema crezca sin comprometer el "núcleo" (kernel) de seguridad.
- **Principio de Seguridad:** Uso de **Sandboxing** (aislamiento) y validación estricta de complementos externos.
- **En resumen:** Es construir una casa donde puedas añadir habitaciones, pero asegurándote de que los nuevos inquilinos no tengan la llave de la caja fuerte.

CONECTIVIDAD

Se refiere a la medida en que el software se comunica con otros sistemas, bases de datos, redes o usuarios a través de internet.

- El Riesgo:** Cada conexión es un punto de entrada potencial. A medida que aumenta la conectividad, crece la **superficie de ataque**. Un software aislado es seguro, pero un software conectado globalmente está expuesto a amenazas constantes de cualquier parte del mundo.

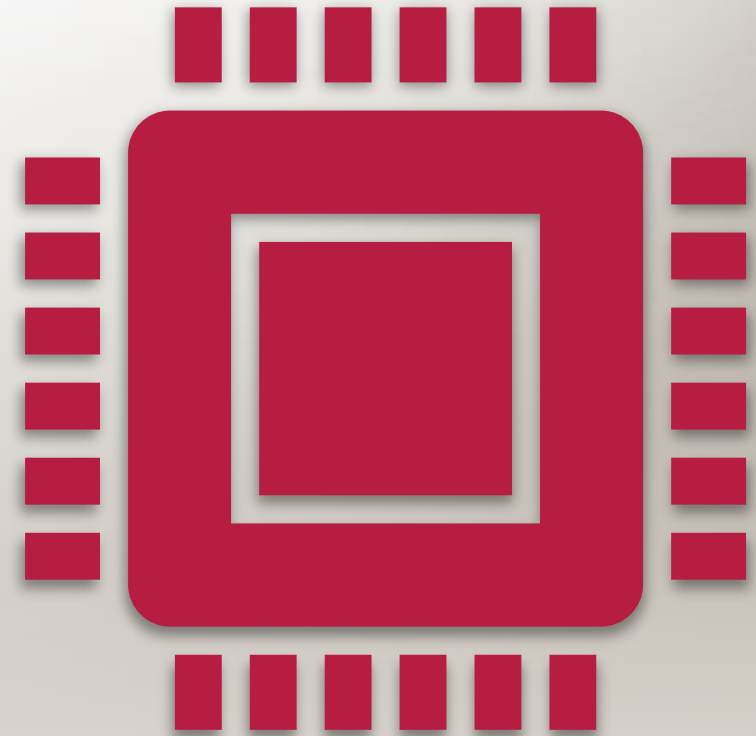
- Principio de Seguridad: Defensa en Profundidad** y el modelo de **Zero Trust** (Nunca confiar, siempre verificar).

- En resumen:** Cuantas más puertas y ventanas tiene un edificio, más guardias necesitas para vigilarlas todas.



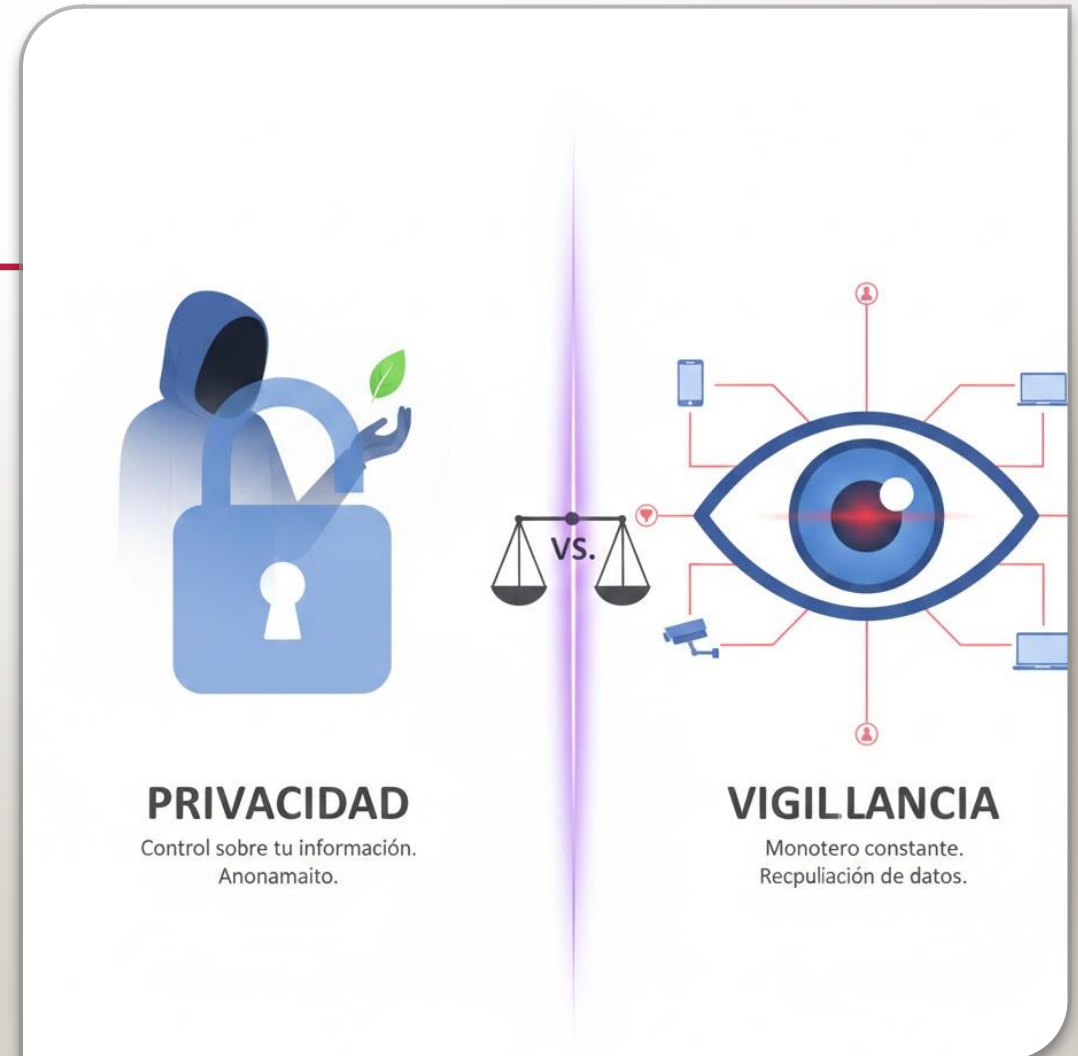
IMPLICACIONES ÉTICAS DE LA SEGURIDAD DEL SOFTWARE

- Hablar de la ética en la seguridad del software es ir más allá de los bits y los bytes; se trata de la responsabilidad que tienen los desarrolladores y empresas sobre la vida y la privacidad de las personas.



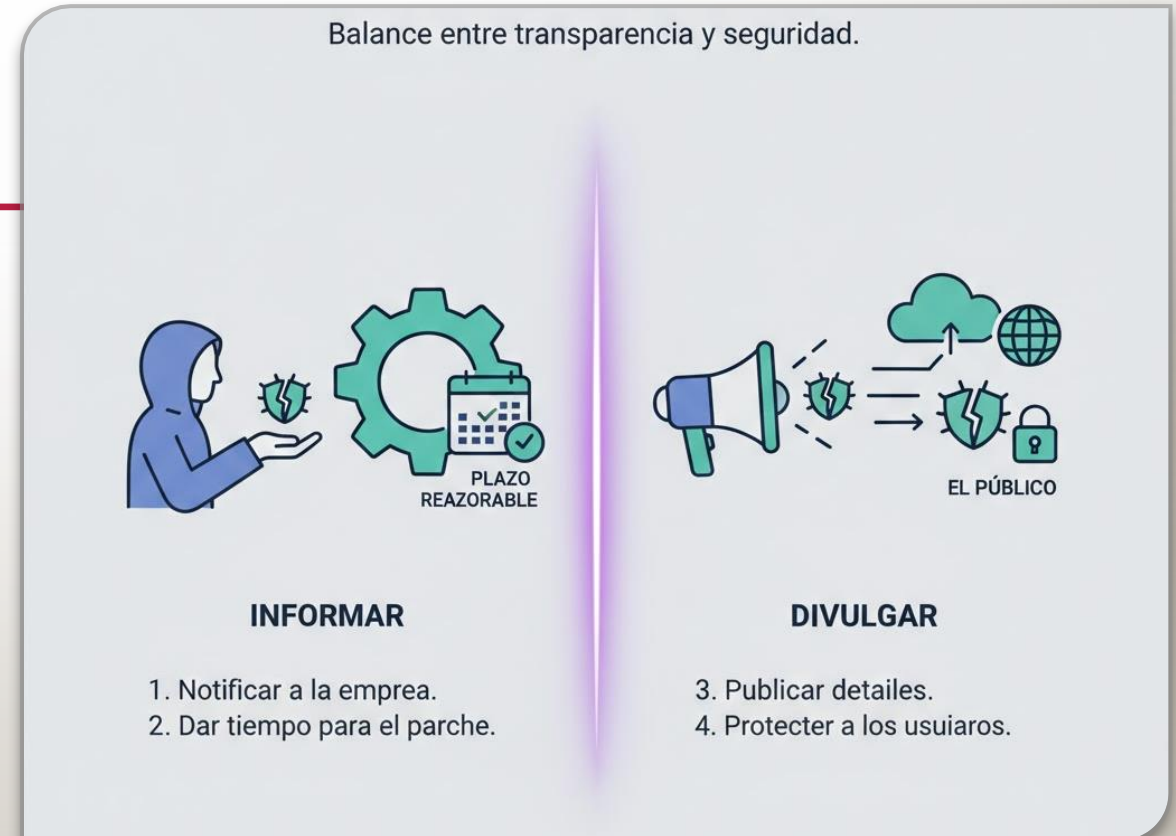
1. PRIVACIDAD VS. VIGILANCIA

- La seguridad a menudo requiere monitorear el comportamiento de los usuarios para detectar anomalías. La línea ética es muy delgada:
- **El dilema:** ¿Cuánta información del usuario es lícito recolectar para "mantenerlo seguro"?
- **Implicación:** El software debe proteger la identidad del usuario sin convertirse en una herramienta de espionaje masivo.



2. DIVULGACIÓN RESPONSABLE (RESPONSIBLE DISCLOSURE)

- Cuando un investigador encuentra una vulnerabilidad, se enfrenta a un problema ético:
- **El dilema:** ¿Debería publicarlo inmediatamente para alertar a todos, o informar primero a la empresa y esperar a que lo arreglen (arriesgándose a que la empresa lo ignore)?
- **Implicación:** La ética dicta que se debe dar un tiempo razonable para el parche antes de hacer pública la falla para evitar el caos.



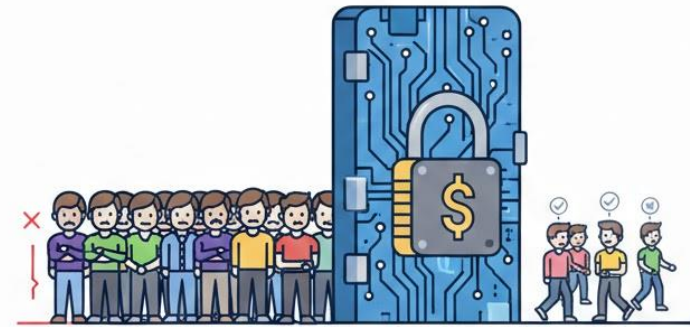
3. SESGOS EN ALGORITMOS DE SEGURIDAD

- Muchos sistemas de seguridad modernos usan Inteligencia Artificial para identificar amenazas.
- **El dilema:** Si el algoritmo aprende de datos sesgados, podría marcar injustamente a ciertos grupos de personas como "sospechosos".
- **Implicación:** La seguridad debe ser **equitativa**. Un software seguro pero discriminatorio es un fracaso ético.



4. EL COSTO DE LA SEGURIDAD Y LA EXCLUSIÓN

- Implementar seguridad de alto nivel es costoso.
- **El dilema:** Si solo los que pagan pueden tener software seguro, la seguridad se convierte en un privilegio de clase.
- **Implicación:** Existe la responsabilidad moral de ofrecer niveles básicos de protección robusta para todos, especialmente en servicios esenciales (salud, banca, educación).



5. RESPONSABILIDAD POR FALLOS (ACCOUNTABILITY)

- Cuando un software inseguro causa daños reales (por ejemplo, en un coche autónomo o un equipo médico):
- **El dilema:** ¿Quién es el responsable?
¿El programador, la empresa o el cliente que no actualizó?
- **Implicación:** La ética exige transparencia sobre las limitaciones del software y una asunción clara de responsabilidades legales y morales.



-
- En seguridad, el "poder" de conocer una vulnerabilidad conlleva la responsabilidad de no usarla para daño personal, un concepto conocido en la comunidad como el **Manifiesto del Hacker Ético**.

LEGISLACIÓN COSTARRICENSE Y OTROS REFERENTES

- En Costa Rica, la seguridad del software y el manejo de datos no se rigen por una sola ley, sino por un ecosistema de normas que han evolucionado para responder a los retos tecnológicos actuales.

1. PROTECCIÓN DE DATOS PERSONALES

Ley N° 8968 (Protección de la Persona frente al Tratamiento de sus Datos Personales):

Establece que los datos pertenecen al ciudadano, no a quien los recolecta.

Consentimiento Informado: Es obligatorio obtener una autorización expresa y por escrito (física o digital) antes de recolectar cualquier dato.

Principio de Seguridad: El artículo 24 obliga a los responsables de bases de datos a adoptar medidas técnicas y organizativas para garantizar la seguridad de los datos y evitar su alteración o acceso no autorizado.

PRODHAB: Es la agencia estatal (Agencia de Protección de Datos de los Habitantes) encargada de vigilar el cumplimiento y sancionar faltas.

Reglamento a la Ley 8968: Detalla los niveles de seguridad (básico, medio, alto) que debe tener el software dependiendo de si maneja datos personales, de acceso restringido o sensibles.

2. DELITOS INFORMÁTICOS Y SEGURIDAD LÓGICA

- Si el software es vulnerado o se usa para fines ilícitos, entramos en el terreno penal.
- **Ley N° 9048 (Reforma de Delitos Informáticos):** Modificó el Código Penal para tipificar delitos específicos:
 - **Suplantación de identidad (Art. 230):** Castiga a quien use identidades falsas en redes o sistemas.
 - **Daño informático (Art. 229 bis):** Sanciona la destrucción o alteración de datos o software.
 - **Sabotaje informático (Art. 229 ter):** Específicamente sobre la interrupción de sistemas.
 - **Instalación de programas maliciosos (Art. 232):** Penaliza la distribución de malware.
 - **Espionaje informático (Art. 231):** Protege los secretos industriales y comerciales almacenados en software.

3. PROPIEDAD INTELECTUAL Y PROTECCIÓN DEL SOFTWARE

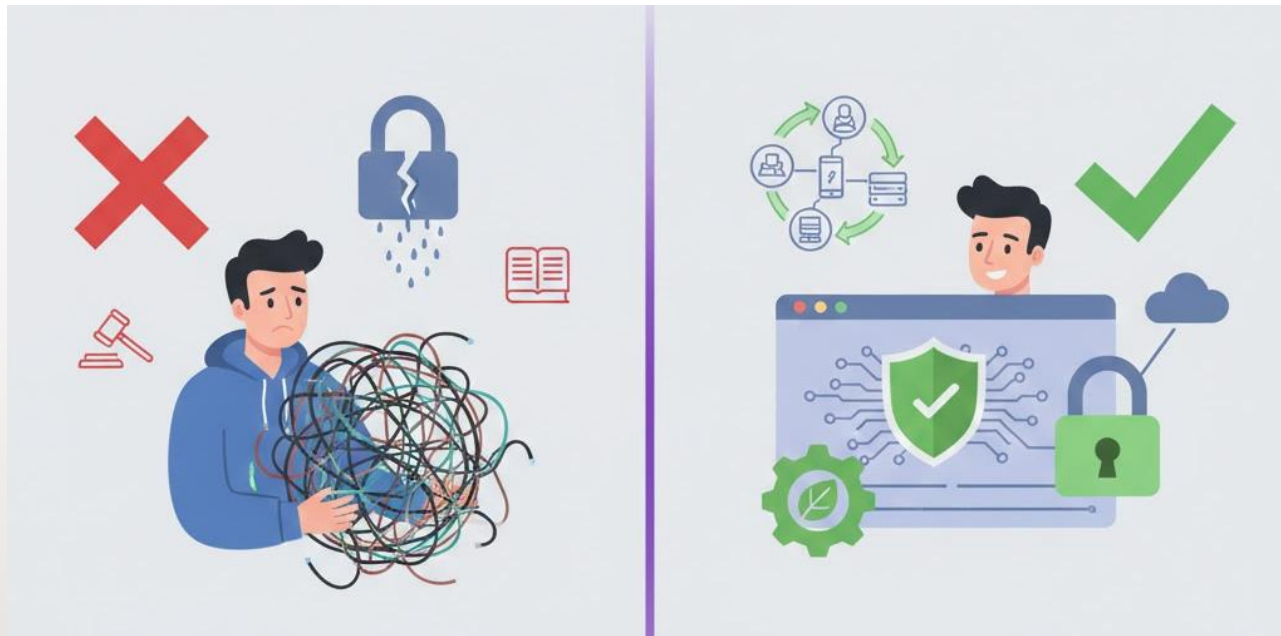
- El software en Costa Rica se protege legalmente no como una invención industrial, sino como una **obra literaria**.
- **Ley N° 6683 (Derechos de Autor y Derechos Conexos):** Protege el código fuente y el código objeto.
 - **Implicación:** El desarrollo de software está protegido desde su creación, pero se recomienda el registro en el **Registro Nacional** para facilitar la defensa ante plagios o usos no autorizados.
- **Ley N° 7975 (Información no Divulgada):** Fundamental para la protección de algoritmos y "know-how" que no se desea hacer públicos, tratándolos como secretos comerciales.

4. NORMATIVA TÉCNICA Y CIBERSEGURIDAD NACIONAL

- Estrategia Nacional de Ciberseguridad: Liderada por el MICITT, establece lineamientos para infraestructuras críticas.
- Ley de Ciberseguridad (En trámite/reciente): Se busca centralizar la respuesta ante incidentes (CSIRT Nacional) y elevar los estándares exigidos a instituciones públicas y proveedores del Estado.
- Estándares Internacionales (ISO/IEC 27001): Aunque son voluntarios, la jurisprudencia administrativa en Costa Rica suele tomarlos como referencia de "buenas prácticas" para determinar si una empresa fue negligente ante una brecha de seguridad.

PUNTOS CRÍTICOS PARA UN DESARROLLADOR O EMPRESA:

1. **Derecho al Olvido:** Los datos personales en bases de datos deben poder ser borrados tras 10 años de su hecho generador (o antes si el fin expiró).
2. **Notificación de Brechas:** Existe la obligación ética (y cada vez más legal) de informar a la PRODHAB y a los afectados si la seguridad del software fue comprometida.
3. **Transferencia Internacional:** Si tu software aloja datos en servidores fuera de Costa Rica (ej. AWS, Azure), debes asegurar que ese país tenga niveles de protección equiparables a los nuestros.



- No diseñes software y luego pienses en la ley. Aplica el concepto de "Privacidad desde el Diseño" (Privacy by Design) para que el cumplimiento normativo esté integrado en el código desde el día uno.

CHECKLIST MAESTRO DE CUMPLIMIENTO NORMATIVO (COSTA RICA) PARA TUS PROYECTOS DE SOFTWARE.

Fase del Proyecto	Requisito Legal / Normativo	Acción Concreta en el Software	Fundamento Legal (CR)
1. Concepción	Privacidad desde el Diseño	Definir solo los datos mínimos necesarios para que el sistema funcione.	Ley 8968, Art. 6
2. Desarrollo	Seguridad de Datos	Implementar cifrado AES-256 en reposo y TLS 1.3 en tránsito.	Ley 8968, Art. 24
3. Interfaz	Consentimiento Informado	Crear un "Pop-up" o casilla de aceptación no marcada por defecto para la recolección de datos.	Ley 8968, Art. 5
4. Usuario	Derechos ARCO	Desarrollar un panel donde el usuario pueda ver, editar o solicitar el borrado de su cuenta.	Ley 8968, Cap. II
5. Arquitectura	Prevención de Delitos	Implementar logs de auditoría inalterables para prevenir accesos indebidos.	Código Penal (Ley 9048)
6. Legal	Políticas de Privacidad	Redactar y publicar Términos y Condiciones claros y accesibles desde el software.	Ley 7472 (Consumidor)
7. Lanzamiento	Propiedad Intelectual	Realizar el depósito de la obra (código fuente) ante el Registro Nacional.	Ley 6683
8. Mantenimiento	Gestión de Brechas	Crear un protocolo de notificación inmediata a la PRODHAB en caso de hackeo.	Reglamento Ley 8968